SETUP (CSVReader.h and CSVReader.cpp)

CODE File Reference: Pages 1 to 2

What's going on here:

- The `CSVReader` is basically pulling temperature data from a CSV file. It opens the file, reads it line by line, and sorts each line into years and temperatures for different countries.
- First, it snags all the country codes from the header. Then it uses these to store the temps in a nested map—years as the main keys and another map for countries with their temps as values.

How I did it:

- Just some basic file I/O and string manipulation to split up the CSV lines.
- There's some error handling thrown in because files seem to be a pain if they don't open right.
- I chose `std::map` because it's good at organizing and accessing data sorted by key (like years and countries here).

Why this way:

- Maps are great for quick lookups and inserts, especially when you need to get data based on year or country later on.

TASK 1: Compute Candlestick Data (CandleStick.h and CandleStick.cpp)

CODE File Reference: Pages 3 to 7

What's going on here:

- The `CandleStick` class is calculating what I call candlestick data for temperatures. This includes the highest, lowest, opening, and closing temps for each year.
- It grabs the data for the whole year, figures out the max and min temps, and calculates the open and close based on the year's start and the previous year's data.

How I did it:

- Highs and lows are straightforward with `max_element` and `min_element`. Opens and closes needed a bit more tweaking using year-start and last-year data.
- I also ended up creating a `PrintCandleStickVector` function to visualize that the `std::vector<Candlestick>` is working as expected.

Why this way:

- Quick way to see if a year was unusually hot or cold or just average.

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    COMMENTS

What would you like to do?
1. Show Candlestick vector for a country
2. Plot Candlestick representation of temperatures of a country
3. Plot within a desired time frame
4. Predict next year's temperature using a 5-year moving average
5. Exit
1
Select a country by entering the corresponding number:
1: AT_temperature
2: BE_temperature
3: BG_temperature
4: CH_temperature
5: CZ_temperature
6: DE_temperature
7: DK_temperature
8: EE_temperature
9: ES_temperature
10: FI_temperature
11: FR_temperature
12: GB_temperature
13: GR_temperature
14: HR_temperature
15: HU_temperature
16: IE_temperature
17: IT_temperature
18: LT_temperature
19: LU_temperature
20: LV_temperature
21: NL_temperature
22: NO_temperature
23: PL_temperature
24: PT_temperature
25: RO_temperature
26: SE_temperature
27: SI_temperature
28: SK_temperature
5
Candlestick data for CZ_temperature:
Year: 1980, Open: -3.422, High: 27.787, Low: -17.001, Close: 5.83945
Year: 1981, Open: 5.83945, High: 27.576, Low: -19.075, Close: 6.98836
Year: 1982, Open: 6.98836, High: 27.797, Low: -17.374, Close: 7.62654
Year: 1983, Open: 7.62654, High: 34.561, Low: -18.411, Close: 8.2258
Year: 1984, Open: 8.2258, High: 33.541, Low: -13.238, Close: 6.97506
```

TASK 2 & 3: Create and Filter a Text-based Plot of the Candlestick Data (PlotCandleStick.h and PlotCandleStick.cpp)

CODE File Reference: Pages 8 to 10

What's going on here:

- This task has required me to plot the candlestick data in text form—using lines and dashes to show temps on a console, which is not the most ideal way to do this but effective.
- Task 3 builds on this by letting you filter which years to plot, so you only see what you really want to.

How I did it:

- Each part of the candlestick (like the high or low temp) gets a spot in a vertical line plot. The plotting function figures out where each temp goes based on its value.
- For filtering, it's just a check if the year is in the range you want before plotting.
- One thing I will mention is my original plan was to do the plotting horizontally rather than vertically how I'm currently doing it. This is because of my time restrictions, so I had to settle here but I'm still meeting the requirements of the assignments.

Why this way:

- Easy and clear—no need for fancy graphs when a simple text layout does the trick. Plus, filtering lets you zoom into whatever time frame you want.

```
What would you like to do?
1. Show Candlestick vector for a country
2. Plot Candlestick representation of temperatures of a country
3. Plot within a desired time frame
4. Predict next year's temperature using a 5-year moving average
5. Exit
3
Enter start year: 2000
Enter end year: 2001
Select a country by entering the corresponding number:
```

```
Candlestick data for AT_temperature:
Year: 1999
     High: |
           |
           |
           |
           |
           |
           |
          -C-
           |
           |
          -O-
           |
           |
           |
     Low:  |

Year: 2000
     High: |
           |
           |
           |
           |
           |
           |
          -C-
          -O-
           |
           |
           |
           |
           |
     Low:  |

What would you like to do?
1. Show Candlestick vector for a country
2. Plot Candlestick representation of temperatures of a country
3. Plot within a desired time frame
4. Predict next year's temperature using a 5-year moving average
5. Exit
```

TASK 4: Predicting Data and Plotting (PredictTemperature.h and PredictTemperature.cpp)

CODE File Reference: Pages 11 to 12

What's going on here:

- I was a bit unsure how to tackle this task. After some research, I could see that this task would be doable by some form of regression or machine learning. But for simplicity sake, I decided to use a 5 period moving average method.
- This task uses the last five years of temps to guess next year's average temperature.
- You pick a country, and it uses the past five years to figure out what's likely to happen next.

How I did it:

- I went with a simple average (the moving average) because our temps tend to follow patterns over years, and this method smooths out the random bad days.
- The math is just an average of averages which is pretty straightforward.

Why this way:

- It’s straightforward and keeps things from getting too complicated. No need for heavy stats or machine learning to get a decent prediction that gives us a ballpark figure.

```
Predictions for AT_temperature:
Year 2014: Actual average temperature: 9.06574°C
Year 2015: Actual average temperature: 9.05732°C
Year 2016: Actual average temperature: 8.39235°C
Year 2017: Actual average temperature: 8.33198°C
Year 2018: Actual average temperature: 9.21341°C
Year 2019: Actual average temperature: 9.19864°C
Based on the data up to year 2019, the predicted average temperature for year 2020 is 8.83874°C.
```

Putting It All Together (Main, WeatherApp.h, and WeatherApp.cpp)

What's going on here:

- The `WeatherApp` file essentially runs the show. It sets up the menu, waits for your choice, and calls the right functions to do what you ask.
- It stitches everything—the CSV reading, calculations, plotting, and predictions—into one smooth operation.

How I did it:

- A loop with a switch-case lets you choose what to do next, and it keeps going until you decide to quit.
- Each case either prints vector data, plots data, or runs predictions.

Why this way:

- Keeps it user-friendly and flexible. You get to decide what you want to see and do, and it’s all interactive right from the console.

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    COMMENTS

sankarshanghosh@Sankarshans-MacBook-Air Weather Application % g++ -std=c++11 *.cpp -o output
sankarshanghosh@Sankarshans-MacBook-Air Weather Application % ./output
What would you like to do?
1. Show Candlestick vector for a country
2. Plot Candlestick representation of temperatures of a country
3. Plot within a desired time frame
4. Predict next year's temperature using a 5-year moving average
5. Exit
```